

Portfolio — Build Prompt

Copy everything below this line into your AI of choice. Bring your own identity, works, and instincts — the prompt describes the *shape* of the thing to build, not the exact contents.

Your job

Build me a single-page, scroll-as-experience personal portfolio in **Next.js (App Router) + TypeScript + Tailwind CSS v4**. The site should behave like a **short film**, not a webpage: a hero statement, a middle beat that says how the person thinks, a dramatic pinned reveal of the work, and a quiet closing section. Motion is the design — not a polish layer added at the end.

Build it in a directory called `portfolio/`. Use **pnpm**. Use **TypeScript everywhere** (no `.js/.jsx`). Single page at `app/page.tsx`. No sub-routes.

Identity (you fill these in)

Gather, then commit to:

- **Name** — will read as two stacked words at viewport scale.
- **Tagline** — one short line. Not a job title. A stance.
- **About line** — one sentence, ~20 words, that you'd actually say out loud.
- **Location, email, resume link, social handles**.
- **Headshot** at `public/headshot.jpg`.
- **Three or four projects** you'd defend to someone whose taste you respect.

If a placeholder is unfilled, leave a clearly-labeled **TODO** in the code rather than inventing content. Inventing content is worse than a missing piece.

Design principles (load-bearing — every decision defers to these)

1. **Confidence through reduction**. Two typefaces, one accent color. No decoration that isn't structural. The refusal to embellish is what reads as expensive.
2. **Scale as drama**. Anything that's meant to land — the name, the section headings — is sized to *impact*, not to fit. Bigness is the message.
3. **Choreographed reveals, not a list**. Sections enter, get pinned, get overtaken. Every scroll position should look composed.
4. **Pauses are part of the composition**. Empty space between beats. Silence makes the next moment land.

5. **No images until they earn it.** Type carries the opening. Project visuals only appear once you've committed to showing the work.

If a decision pulls against any of these, the principle wins.

Tech stack (exact)

- **Next.js 15** (App Router) + React 19 + TypeScript 5
- **Tailwind CSS v4** (using `@import "tailwindcss";` and `@theme` inline block — no `tailwind.config.js`)
- **motion** package (formerly Framer Motion) — import from `motion/react`
- **lenis** for smooth scrolling
- **next/font/google** for the two typefaces. No local font files.
- **next/image** for all images.
- No GSAP. No other animation libraries. No CSS-in-JS libraries.

Hard rules

- All color via CSS variables — never hardcode a hex in a component.
 - Tailwind utility classes preferred; inline `style={}` is OK only for dynamic motion values (transforms, opacity tied to scroll) and for size clamps that can't be expressed as utilities.
 - A single shared easing curve (something soft and out-only, e.g. `[0.22, 1, 0.36, 1]`) is reused everywhere. Define it as a constant; don't sprinkle bezier tuples.
 - The email and resume URL live as constants (`EMAIL`, `RESUME_URL`). Don't sprinkle string literals.
-

Visual system

Color

Pick a **restrained palette**: a background, a foreground, a muted foreground, an accent, and a hairline-rule color. Five tokens. That's the whole site.

The mood matters more than the exact hexes. Aim for: a warm off-white background, a near-black foreground with a slight undertone (not pure `#000`), and an accent that lives somewhere in the same family as the foreground rather than fighting it. The accent should feel like it *belongs* to the foreground — a deeper, more saturated cousin — not a contrasting brand color.

Declare them as CSS variables in `app/globals.css` and expose them to Tailwind v4 via an `@theme` inline block so utilities like `text-fg`, `bg-bg`, `text-fg-muted`, `border-rule`, `text-accent` exist.

Typography

Two typefaces, no exceptions:

- **One display serif** for moments of impact: the hero name, the giant section headings, project titles, the closing line. Should be high-contrast, slightly editorial, comfortable at very large sizes. (Bodoni Moda, Playfair Display, GT Super, or similar.) Use it at weight 500, tight tracking ($-0.02em$), tight leading ($0.82-0.9$) when set large. Never use it below $\sim 40px$.
- **One geometric sans** for everything else: body, nav, captions, footer, link handles. (Montserrat, Inter, GT Walsheim, or similar.)

Load both via `next/font/google` in `app/layout.tsx`. Expose as CSS variables and wire them into Tailwind utilities (`font-display`, `font-body`). Body inherits the sans.

For small all-caps labels (category tags, footer copyright): the body font at weight 500, uppercase, wide tracking ($\sim 0.18em$), $\sim 11px$. Use `tabular-nums` anywhere numerals appear.

Motion language

- **One easing curve everywhere**, soft and out-only.
- **Durations:** 600–900ms for entrances; 200–300ms for hover and micro-feedback.
- **Distances:** small for non-scroll-bound moves (20–120px). Scroll-bound moves can travel much farther — that's the point of pinning.
- **No bounces, springs, overshoots, or elastic curves.** The site is poised, not playful.

Layout

- No explicit grid. $\sim 6vw$ side gutters (`px-[6vw]`).
 - Generous vertical space. Empty space is content.
-

Persistent shell

`app/layout.tsx`

Load the two typefaces via `next/font/google` and expose them as CSS variables. Wrap `{children}` in a `<SmoothScroll>` client component. Metadata: `title` = your name; `description` = your tagline.

`components/SmoothScroll.tsx`

Client component. On mount, instantiate `Lenis` with an eased duration ($\sim 1.2s$) and a soft easing function, drive its `raf` loop with `requestAnimationFrame`, and clean up on unmount. Render `{children}` straight through. Wheel scroll should feel *eased*, not snappy.

Page composition

Four acts, in order, in `app/page.tsx`. Nothing else.

```
<main>
  <Act1 /> { /* statement */ }
  <Act2 /> { /* thesis */ }
  <Act3 /> { /* work */ }
  <Act4 /> { /* quiet exit */ }
</main>
```

Act 1 — Statement

A full-viewport opening. The visitor's first impression is your name at scale — nothing else competes for attention.

What's on the screen

- **Your name, huge.** Display serif, viewport-scaled (something like `c lamp(64px, 12vw, 180px)`), set as two stacked or side-by-side words. The name occupies most of the screen height.
- **A hairline rule** below the name, drawn full-width, separating the statement from the small print beneath it.
- **Below the rule:** a tagline on the left, a row of icon links on the right (resume, email, social handles).
- **Top-right nav:** two stacked links — one to the work, one to the about. Tiny. They quietly anchor the corner without distracting from the name.

Choreography

The name should **arrive**, not appear. Split into per-letter spans, each inside its own `overflow: hidden` clip, with the letter rising from below its mask. Stagger the letters by a small constant (~30ms) so the word builds left-to-right; offset the second word so it follows the first by a beat.

The hairline rule scales in from the left (`scaleX: 0 → 1`) once the name has mostly landed. The tagline and icon row fade up after the rule — a few short staggered delays, nothing elaborate.

The nav links slide in from the right and fade in last. On hover, the whole link translates a few pixels to the right with a soft transition.

Nav scroll behavior

Clicking a nav link should scroll to a computed position, not just `scrollIntoView`. For each item, store a target section id, a scroll-progress fraction *within* that section, and a pixel offset. Compute `top = sectionTop + sectionHeight * scrollProgress + offset`, then `window.scrollTo({ top, behavior: 'smooth' })`. Update the URL hash via `history.replaceState` so the address bar reflects the location without triggering its own scroll.

For sections that pin (like the work act), the scroll-progress fraction matters: you want the user to land *inside* the pinned moment, not at its top edge where nothing's resolved yet.

Email copy interaction

The email icon (and the email handle in Act 4) is a button, not a `mailto:` link. On click, copy the address to the clipboard and swap the icon for the word **copied** for ~1.5s using `<AnimatePresence mode="wait">`. The swap is a small fade + few-pixel y-shift, same easing as everything else.

Act 2 — Thesis

The bridge between the hero and the work. Before showing *what you've made*, show *how you see yourself* — distilled to its sharpest form. The goal of this act is **a single visual idea, held long enough to land, then cleared away**.

The shape of the idea

Pick a **duality that defines you**: two words that, held in tension, describe how you work. Two roles, two instincts, two halves of one practice. (CREATOR / CODER, WRITER / ENGINEER, DESIGNER / BUILDER, STRATEGY / CRAFT.) One word alone is a label; two in tension is a thesis.

Give each word one sentence of context. Not a bio. Not a list of skills. A short statement of what that word means to *you* specifically.

The choreography (essence, not recipe)

The section pins for a long scroll budget (a few viewports tall, sticky inner stage). Across that scroll, three things happen in order:

1. **A divider appears.** Some single graphic element — a line, a slash, a seam — draws itself across the stage and splits it. The "tension" between the two words made literal. One decisive gesture, not an animation reel.
2. **The two halves reveal their content.** Each half wipes, fades, or otherwise resolves into legibility on its own side of the divider. The word is huge; the sentence is small and quiet. Asymmetric placement — one half leads with the word high, the other leads with the sentence low. Mirror symmetry reads as a diagram; asymmetry reads as composed.
3. **The stage clears.** Continued scroll slides the two halves off in opposite directions. The screen goes empty for a beat before Act 3 takes over. The empty beat matters.

All three phases driven by a single `useScroll` progress value, with each phase claiming a non-overlapping window. No `whileInView` triggers — everything is scrub-bound so the user controls the pace.

Principles to hold onto

- **One idea, fully committed.** No third panel, no tagline above, no CTA below.
- **Display type at viewport scale.** If the words feel comfortable, they're too small.
- **The divider is a character.** Whatever shape you choose, it should look intentional and

singular.

- **Pinning earns its keep with motion.** If you pin, *something* must change across the scroll, or the user feels stuck.
- **Mobile is the same idea, rotated.** Left/right split becomes top/bottom split. Divider axis flips, exit direction flips with it, type scales up.

What to avoid

- A bulleted "values" or "principles" list. That's a résumé, not a thesis.
 - Three or more beats. Two concentrates; three dilutes.
 - Decorative motion that doesn't carry the idea (sparkles, parallax dust, animated gradients behind the words).
 - Symmetric, centered, mirror-image layouts.
-

Act 3 — Work

The most important section. The pinned reveal where your work earns the rest of the page. The structure is fixed; the spectacle is in the proportions.

The shape

A position: sticky viewport inside a very tall outer section (around 8–10 viewport heights of scroll). Inside the pinned stage:

- **A heading larger than the screen** — the word **WORK** (or your own equivalent: **PROJECTS**, **MADE**, **CASES**), set in the display serif at something like 32vw. So large that its letters span edge-to-edge across the viewport. The heading acts as a backdrop; it sits behind the cards at reduced opacity so the cards remain the focal point.
- **Project cards traveling over the heading.** Each card scroll-scrubs from below the viewport, up through, and out the top — like film cells passing in front of a fixed title card.

The letter choreography

The letters of the heading don't sit there from the start. As the user scrolls into the section, each letter slides in from offscreen — some from the left, some from the right — over its own scroll window. Stagger the windows so they arrive in sequence, not all at once. Once a letter has landed it stays put for the rest of the section.

Each letter is its own `motion.span` with an `x` transform driven by `useTransform(progress, [start, end], [from === "right" ? "120vw" : "-120vw", "0vw"], { clamp: true })`. Opacity rises to ~55% (not 100%) so the cards passing over it stay readable.

The cards

Three or four cards, alternating sides (left, right, left, right). Each card:

- A scroll-bound `y` transform that takes it from below the viewport (~~`115vh`~~) all the way through and out the top (`-80vh`). The card's "active" window inside the section overlaps with adjacent cards — as one is leaving the top, the next is rising from below.
- Width around half the viewport (`48vw`), capped at a sensible max (`~640px`). Positioned absolutely against the left or right gutter.
- Each card is a `motion.a` linking to the live URL (or repo, whichever is the canonical home).

Card visual:

- A 4:3 image container with a generous border radius (`~24px`), a soft shadow, and a subtle hover lift (`scale: 1.02, y: -8, ~400ms ease`).
- The image uses `next/image` with `fill` and `object-cover`.
- Below the image, a caption row: project number (e.g. `01`), title in the display serif, category tag in small all-caps on the far side.

Tune the scroll windows so cards don't collide visually — there should be one card dominant on screen at a time, with the next already rising in.

View-all (optional)

Near the end of the section, a pill fades in centered over the heading. Currently links to `#` — placeholder for a future `/work` route. Subtle border, soft shadow, hover lifts the shadow. Content: `View all (N)` where `N` is the total count of works you'd want to surface there.

What to avoid

- A grid of project tiles. That's a portfolio website. This is a film.
- Visible scroll indicators, progress bars, or "scroll down" text inside this section. The motion itself is the indicator.
- Cards that all enter from the same side. Alternation creates rhythm.
- A heading that fits the viewport. If it fits, it's not big enough.

Act 4 — Quiet exit

After the spectacle, the page goes quiet. No scroll-pinning, no scroll-bound transforms — just a single composed fade-up.

What's on the screen

Centered vertically, max-width around `65ch`, in this order:

1. **A circular headshot.** Cropped tight, sized between `140px` and `220px` depending on viewport. Soft shadow. A slight desaturation / contrast filter (`saturate(0.85) contrast(1.02)`) so the photo sits in the same tonal world as the rest of the page rather

than punching through it.

2. **The about line.** One sentence. The body font at medium weight, slightly larger than body text (~1.1–1.4rem), comfortable leading. Not a paragraph. Not a list.
3. **A link grid.** All the contact points in one row (or two-column grid on mobile): resume, email (the same MailCopy button as Act 1), and each social handle as icon + handle text. Muted by default; foreground on hover.
4. **A footer line.** Tiny, uppercase, wide-tracked: © <name> · <city, state>.

Motion

A single staggered fade-up on view. Container variant with `staggerChildren(0.11s)` and a small `delayChildren(0.25s)`. Each child fades up from `y: 16` over ~750ms with the shared easing.

Respect `useReducedMotion()` — if reduced, skip the variants entirely and render the final state.

Anchor / IDs

Decide upfront which act owns `#about` — the nav links to one of them. If Act 2 is your "who I am" beat, give it the anchor; if Act 4 is, give it there instead. Don't ship with two elements sharing the same id.

Inline SVG icons

Don't pull in an icon library. Hand-write the few SVGs you need:

- **Resume:** a simple document outline. Stroke `currentColor`, no fill, 1.5 stroke width.
- **Mail:** a rectangle with a triangle inside (the envelope flap). Same stroke treatment.
- **GitHub, LinkedIn, X, Instagram:** the canonical single-path brand glyphs, `fill="currentColor"`, sized 18×18 (X at 16×16).

Reuse the exact same SVG markup in Act 1 and Act 4. All icons inherit color from text, so `text-fg-muted` parent → muted icon, `hover:text-fg` → emphasized icon.

Data

Create `lib/projects.ts`. Type and example shape:

```
export type Project = {
  id: string; // "01", "02", ...
  title: string;
  category: string; // short tag, e.g. "AI Interview Prep"
  description: string; // one short sentence
  tags: string[]; // ["Next.js", "OpenAI", "Postgres"]
  screenshot: string; // "/something.png"
```

```

links: { live?: string; github?: string };
};

export const projects: Project[] = [
  // 3-4 projects you'd defend
];

```

Screenshot files go in `public/`. Reference them by their `/filename.png` paths (URL-encode spaces as `%20`).

File tree to produce

```

portfolio/
├── app/
│   ├── globals.css           # color tokens, @theme inline, base font on body
│   ├── layout.tsx           # next/font, SmoothScroll wrapper, metadata
│   └── page.tsx             # <Act1 /> <Act2 /> <Act3 /> <Act4 />
├── components/
│   ├── SmoothScroll.tsx
│   ├── Nav.tsx
│   ├── AnimatedName.tsx
│   └── acts/
│       ├── Act1.tsx
│       ├── Act2.tsx
│       ├── Act3.tsx
│       └── Act4.tsx
├── lib/
│   └── projects.ts
├── public/
│   ├── headshot.jpg         # I provide
│   └── *.png                 # project screenshots I provide
├── package.json
├── tsconfig.json
├── postcss.config.mjs       # { plugins: { "@tailwindcss/postcss": {} } }
├── next.config.ts
└── next-env.d.ts

```

Build order

Build in this order. Don't move forward until the current step *feels right* — taste is the gate.

1. **Scaffold** Next.js 15 + TypeScript + Tailwind v4 in `portfolio/`. Verify `pnpm dev` runs an empty page.
2. **Tokens + fonts**. Write `globals.css` with the color variables and `@theme inline`. Wire the two typefaces into `layout.tsx` via `next/font/google`. Body inherits the sans.
3. **Smooth scroll shell**. Build `SmoothScroll.tsx` and wrap `{children}` in `layout.tsx`. Test that wheel scrolling feels eased.
4. **Act 1**. Build the animated name first — verify the per-letter mask reveal feels right at full viewport. Then nav, then the rule + tagline + icon row. Wire the `MailCopy` interaction. **Do not proceed** until Act 1 feels composed.

5. **Act 2.** The duality + the divider + the clear-the-stage exit. Tune the three scroll windows until each phase reads cleanly.
 6. **Act 3 — letters first.** Build the sticky tall shell and get the heading letters sliding in. Don't add cards yet. Verify the choreography on a slow scroll.
 7. **Act 3 — cards.** Add the project cards traveling over the pinned heading. Tune the per-card scroll windows if cards collide or feel crowded. Add the view-all pill last.
 8. **Act 4.** Centered headshot + about + link grid + footer with staggered fade-up. Respect `useReducedMotion`.
 9. **Polish.** Focus states (visible ring using the accent color), OG metadata, favicon (`app/icon.png`), perf check (`pnpm build`).
-

Things NOT to do

- Don't add an "About" / "Experience" / "Skills" / "Services" section. The about line in Act 4 is the entire bio. The duality in Act 2 is the entire pitch.
 - Don't add sub-routes. Everything is one scroll.
 - Don't introduce a second accent color, gradient backgrounds, glows, noise overlays, or grain.
 - Don't use a monospace font anywhere visible. `tabular-nums` for numerals if needed.
 - Don't use GSAP, ScrollTrigger, react-three-fiber, or any 3D library.
 - Don't hardcode hex colors in components. CSS variables via Tailwind utilities.
 - Don't replace `motion/react` with `framer-motion`. The package is now `motion`.
 - Don't add dark mode. Pick one palette, commit to it. (If the palette is dark, fine — but don't ship a toggle.)
 - Don't add a cookie banner, analytics modal, or "subscribe" prompt. This is a portfolio, not a funnel.
-

When you're done

Print a short summary of:

- What I need to add to `public/` (headshot + project screenshots, with filenames matching `lib/projects.ts`).
- Any placeholders left as `TODO` that need my input.
- The exact command to run the dev server.

That's it. Build.